Add cover

Issue Report: EKS Cluster Creation Failure in Shared VPC Due to Untagged Resource Cleanup

1. Environment Details

AWS Account (Application Account):
`998882071977`

VPC ID:
`vpc-0c083e654bac0e84a`

VPC Owner Account:
`743408735039` (Shared via AWS RAM)

This indicates that the VPC is part of a **Shared VPC architecture**, where the VPC is owned by a centralized networking account and shared with the application account.

---

2. EKS Cluster Deployment Method

The EKS cluster is created using the **Terraform automation API exposed through Swagger UI**.

API Documentation:
https://ansibleplus.verizon.com/cloud/apidocs/#/Terraform%20Common/api.terraform_common.createStack

Parameters used

Parameter	Value
Cloud Provider	aws
Region	us-east-1
Account Name	vz-itnp-k20v-cnsa2
Stack Name	eks-pagidhe-001-k0xv-npge
VSAD	K0XV

Request Body Used

```
{
  "GitRepoUrl": "git@gitlab.verizon.com:eks/2.0.git",
  "GitBranch": "feature/eks-terraform",
  "TemplDir": "eks-terraform-2.0/",
  "OwnerId": "pagidhe",
  "StackAction": "CreateOrUpdate",
  "StackProtection": "Off",
  "TfVersion": "Terraform_1.5.x",
  "TfRefresh": true,
  "TfDynamicParams": {},
  "TfVarsFile": "terraform.tfvars.json",
  "PlanOnly": false,
  "clientContext": {
    "clientUrl": "https://jenkins-gts.vpc.verizon.com/gts/job/GTS.H9VV.AIS.Team/job/ITCS.H9VV.AIS.SomeJob/525/"
  }
}
```

The Terraform job consumes variables from **terraform.tfvars.json**, which includes:

- EKS version
- tags
- subnet IDs
- VPC ID
- networking configuration

No networking resources are created by Terraform because networking is managed centrally.

---

3. Terraform Security Scan Limitation

During the Terraform pipeline execution, a **security scan check fails**.

Failed Check

```
CKV_AWS_339
Ensure EKS clusters run on a supported Kubernetes version
```

Observed Behavior

EKS Version	Result
1.34	Scan Fails
1.33	Scan Passes

Root Cause

The security scanner currently only recognizes **EKS versions up to 1.33** as supported.

However, AWS currently supports newer versions such as **1.34**.

Required Change

The scanner rule should be updated to allow:

```
EKS version >= 1.34
```

or updated dynamically based on AWS supported versions.

---

4. Additional Issue: Cleanup Lambda Removing Untagged Resources

During EKS cluster creation another issue was observed.

A **cleanup Lambda function** exists in the shared networking environment which **automatically deletes resources that do not have tags**.

This cleanup occurs **almost instantly**.

---

5. EKS Cluster Creation Behavior

When Terraform creates an EKS cluster, AWS internally creates certain resources automatically.

One of these is the **cluster security group**.

Example name pattern:

```
eks-cluster-sg-<cluster-name>-<random>
```

These resources are created by **AWS EKS service itself**, not Terraform.

Initially these resources may appear **without tags for a short duration**.

---

6. Failure Sequence

The failure occurs due to the following sequence:

```
Terraform → Create EKS cluster
        ↓
AWS → Creates cluster security group
        ↓
Cleanup Lambda → detects untagged resource
        ↓
Cleanup Lambda → deletes security group
        ↓
EKS cluster creation fails
```

Because the **security group is deleted immediately**, the cluster provisioning cannot complete.

---

7. Likely Lambda Trigger Mechanism

Since the deletion happens almost instantly, the Lambda function is likely triggered by one of the following:

- EventBridge rule
- CloudTrail event trigger
- Very short scheduled execution

This strict behavior causes **legitimate AWS-managed resources to be deleted prematurely**.

---

8. IAM Role Used for EKS Cluster Creation

The EKS cluster is created using the following role:

```
arn:aws:iam::998882071977:role/Vz-ServiceRoleForAmazonEKS
```

This role is assumed during cluster provisioning.

---

9. Recommended Solutions

Solution 1 (Recommended)

Whitelist resources created by the **EKS service**.

Possible conditions:

Allow resources created by:

```
eks.amazonaws.com
```

or

```
arn:aws:iam::998882071977:role/Vz-ServiceRoleForAmazonEKS
```

Solution 2

Ignore resources containing the tag:

```
aws:eks:cluster-name
```

These tags are automatically applied by AWS for EKS resources.

Solution 3

Ignore security groups matching the naming pattern:

```
eks-cluster-sg-
```

These are automatically generated by AWS during cluster provisioning.

Solution 4 (Best Practice)

Introduce a **grace period before deleting untagged resources**.
Example logic:

```
If resource has no tags AND resource age > 60 minutes → delete
```

This gives AWS services sufficient time to attach tags.

---

10. Summary

Two issues are currently impacting EKS cluster provisioning:

1️⃣ Security Scanner Limitation

The Terraform scanner only allows EKS versions **≤ 1.33**.
This should be updated to allow newer supported versions.

2️⃣ Untagged Resource Cleanup

The cleanup Lambda deletes AWS-generated EKS resources before tags are applied, which causes cluster creation to fail.

---

11. Required Actions

1. Update the security scanner rule to support **EKS version 1.34 and above**.
2. Modify the cleanup Lambda to:
   - Ignore resources created by **EKS**
   - Ignore security groups with prefix **eks-cluster-sg-**
   - Allow a **grace period before deleting untagged resources**.

If implemented, these changes will allow **EKS clusters to be provisioned successfully in the shared VPC environment** while still maintaining tagging compliance policies.